SEGURANÇA DA INFORMAÇÃO / GBC083

# Rainbow Tables

## Implementação, testes e análise experimental com senhas Diceware

Matheus Dantas Santana

Vanessa Dias Ferreira

Professor Ivan Sendin

Universidade Federal de Uberlândia, 25 de outubro de 2026

# Objetivo e espaço de senhas

**7.776**

**senhas possíveis**

Cada senha é uma entrada da lista Diceware original, sem o código numérico e sem quebra de linha.

Combinações de duas palavras não foram avaliadas.

O experimento compara tabelas altas e estreitas com tabelas baixas e largas.

Medimos a proporção de senhas recuperadas, o tempo das consultas e o tamanho dos pares guardados.

Uma redução fixa funciona como comparação para observar o efeito das cores.

# A cadeia guarda só os extremos

Exemplo real com três passos de hash e redução, começando na entrada **a**.




## Par armazenado

```
("a", "opt")
```

As palavras intermediárias são recalculadas quando uma busca precisa conferir a cadeia.

## O papel das cores

H usa SHA-256 com o prefixo GBC083. R escolhe uma palavra a partir do hash e da coluna.

Nessa linha, os candidatos comparados por H são a, iowa e pork. O extremo opt isolado não acrescenta cobertura.

# Busca e confirmação do candidato

1. Receber o hash e testar em qual coluna da cadeia ele pode estar.
2. Continuar até um final e consultar o índice de pares.
3. Refazer cada cadeia candidata e comparar seus hashes com o alvo.

## O final não é uma prova

Vários inícios podem levar ao mesmo final. O índice preserva todos eles.

A senha só é devolvida depois de confirmar seu hash. Um candidato incorreto não encerra a busca.

---

No exemplo, consultar H("iowa") na cadeia a–opt recupera iowa. Não há varredura extra da lista para completar falhas.

# Experimento e computador utilizado

| Parâmetro | Valor |
|---|---|
| Formatos m×t | 512×16, 256×32, 128×64, 64×128 |
| Sementes da tabela | 17, 43 e 91 |
| Alvos por cenário | 200, com semente 2026 |
| Modos | Cores e redução fixa |

Os mesmos alvos, sorteados independentemente da construção, são usados nos 24 cenários. São 4.800 consultas.

| Componente | Configuração |
|---|---|
| Processador | Ryzen 7 5700X, 8 núcleos / 16 processadores lógicos |
| Memória RAM | 16 GiB |
| Sistema | Windows 11 |
| Python | 3.14.7 |

Todos os formatos fazem 8.192 passos na construção. Repetições impedem garantir cobertura integral só por essa contagem.

# Verificações funcionais

| Caso | Resultado |
| --- | --- |
| Hash do exemplo do professor | Valor esperado confirmado |
| Inícios com final repetido | Ambos preservados no índice |
| Hashes presentes nas cadeias | 80 de 80 recuperados |
| Alvo fora da cobertura | None, sem senha incorreta |

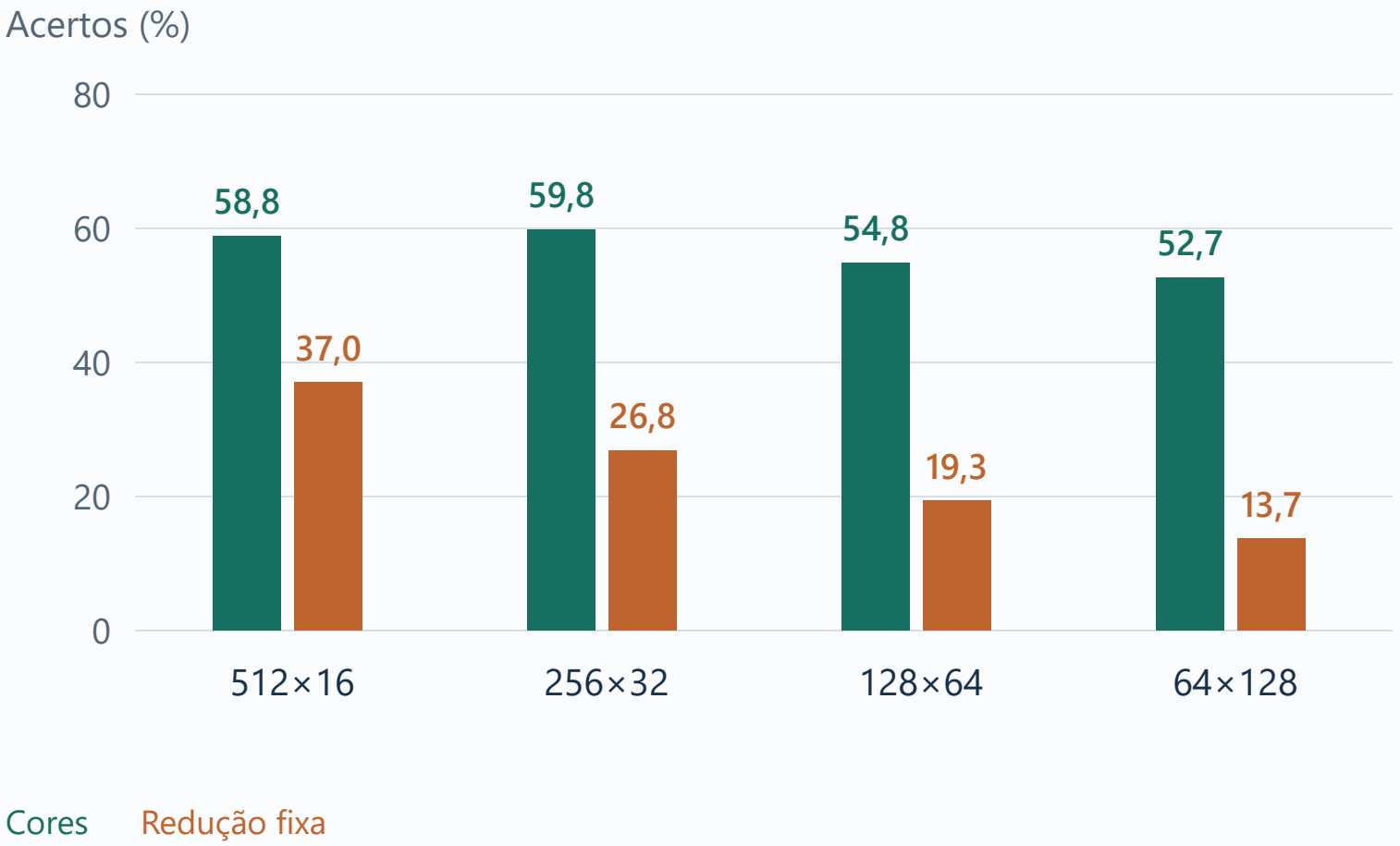
## O que foi conferido

As 80 consultas percorrem todas as cinco posições de oito cadeias, nos dois modos.

O caso ausente usa largura 1: escolhemos uma entrada que não está entre os inícios da tabela.

O código completo dessas verificações está no apêndice. Esses casos de correção não são usados para estimar eficácia.

# Recuperação dos alvos




**52,7% a 59,8%**

de recuperação média com cores.

A redução fixa ficou entre 13,7% e 37,0%. Os valores são médias das três sementes.

A amostra mede recuperação de 200 alvos. Ela não mede a cobertura exata das 7.776 entradas.

# Eficácia e armazenamento

| Modo | Formato | Acertos / 200 | Taxa (%) | Finais distintos | Disco (bytes) |
|---|---|---|---|---|---|
| cores | 512×16 | 117,7 | 58,8 | 328,0 | 8.430 |
| cores | 256×32 | 119,7 | 59,8 | 160,7 | 4.249 |
| cores | 128×64 | 109,7 | 54,8 | 80,7 | 2.101 |
| cores | 64×128 | 105,3 | 52,7 | 40,3 | 1.058 |
| fixa | 512×16 | 74,0 | 37,0 | 314,0 | 8.436 |
| fixa | 256×32 | 53,7 | 26,8 | 147,0 | 4.175 |
| fixa | 128×64 | 38,7 | 19,3 | 73,3 | 2.102 |
| fixa | 64×128 | 27,3 | 13,7 | 38,0 | 1.067 |

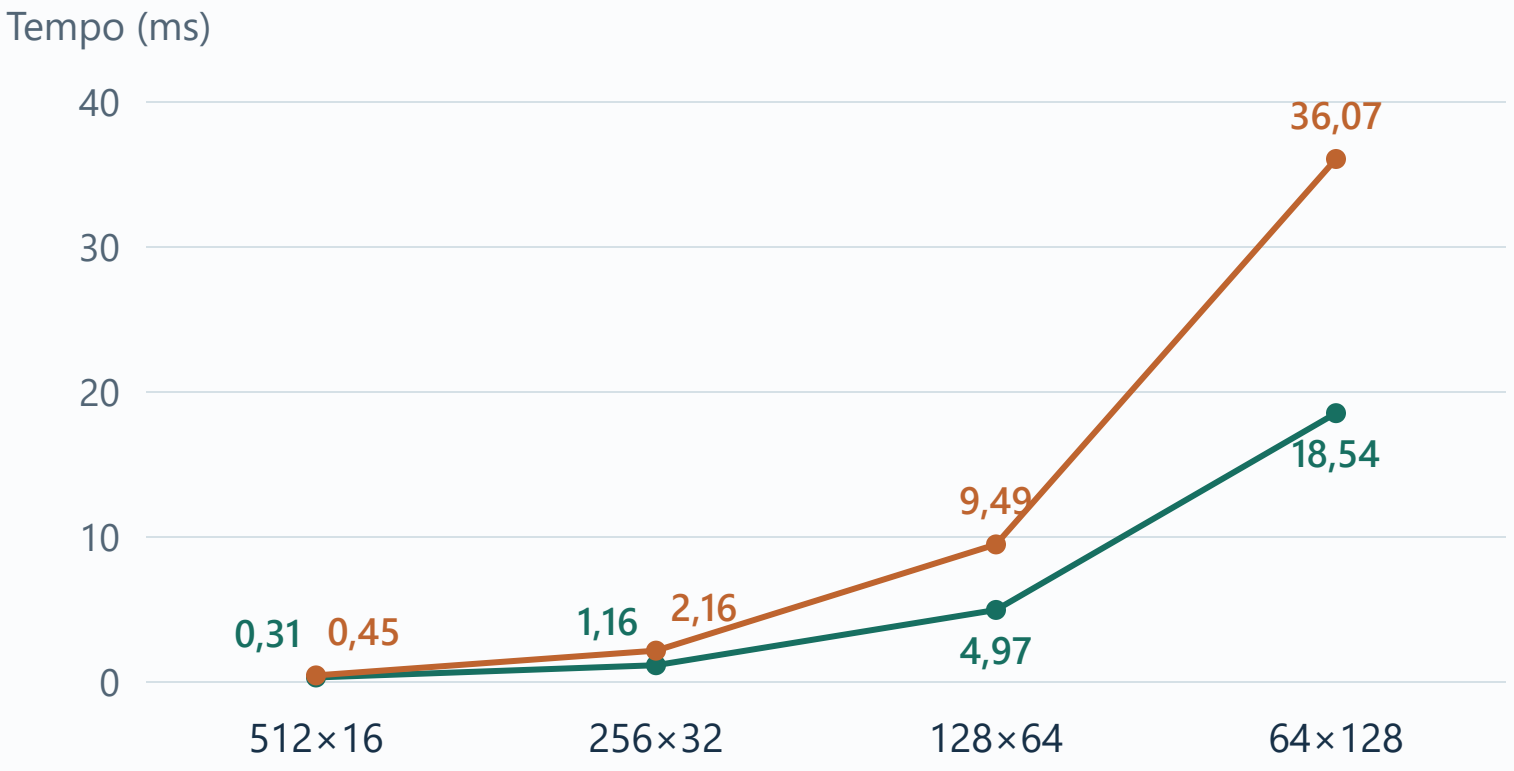
Médias das três sementes. Disco é o tamanho do JSON dos pares início/fim. A RAM do processo não foi medida. Finais distintos não indicam, sozinhos, a cobertura da tabela.

# Variação entre as sementes

| Formato | Semente | Acertos cores | Cores (%) | Acertos fixa | Fixa (%) |
|---|---|---|---|---|---|
| 512×16 | 17 | 121 | 60,5 | 68 | 34,0 |
| 512×16 | 43 | 122 | 61,0 | 79 | 39,5 |
| 512×16 | 91 | 110 | 55,0 | 75 | 37,5 |
| 256×32 | 17 | 117 | 58,5 | 47 | 23,5 |
| 256×32 | 43 | 132 | 66,0 | 61 | 30,5 |
| 256×32 | 91 | 110 | 55,0 | 53 | 26,5 |
| 128×64 | 17 | 117 | 58,5 | 35 | 17,5 |
| 128×64 | 43 | 122 | 61,0 | 41 | 20,5 |
| 128×64 | 91 | 90 | 45,0 | 40 | 20,0 |
| 64×128 | 17 | 105 | 52,5 | 25 | 12,5 |
| 64×128 | 43 | 104 | 52,0 | 28 | 14,0 |
| 64×128 | 91 | 107 | 53,5 | 29 | 14,5 |

Em cores, 256×32 variou de 55% a 66%, e 512×16 de 55% a 61%. A diferença entre as médias 59,8% e 58,8% não demonstra superioridade geral de um formato.

# Tempo médio por consulta




**59,5×**

mais tempo por consulta em cores ao passar de 512×16 para 64×128.

Cadeias longas exigem testar mais posições e recalcular trechos maiores. Falhas esgotam as hipóteses, enquanto acertos podem encerrar a busca antes.

# Custos de construção e de busca

| Modo | Formato | Construção (ms) | Índice (ms) | Total buscas (s) | Consulta (ms) | Sucesso (ms) | Falha (ms) |
|---|---|---|---|---|---|---|---|
| cores | 512×16 | 15,09 | 0,900 | 0,062 | 0,31 | 0,18 | 0,50 |
| cores | 256×32 | 15,05 | 0,040 | 0,231 | 1,16 | 0,64 | 1,92 |
| cores | 128×64 | 15,16 | 0,025 | 0,994 | 4,97 | 2,50 | 7,92 |
| cores | 64×128 | 14,96 | 0,016 | 3,708 | 18,54 | 7,12 | 31,25 |
| fixa | 512×16 | 15,15 | 0,070 | 0,091 | 0,45 | 0,25 | 0,58 |
| fixa | 256×32 | 14,92 | 0,043 | 0,433 | 2,16 | 1,07 | 2,56 |
| fixa | 128×64 | 14,85 | 0,023 | 1,898 | 9,49 | 3,71 | 10,88 |
| fixa | 64×128 | 14,83 | 0,014 | 7,214 | 36,07 | 8,31 | 40,47 |

Médias das três sementes. Total soma 200 buscas por tabela, e consulta é total/200. Sucesso e falha são médias das médias condicionais de cada semente. Construção e índice são custos iniciais, medidos separadamente.

# Análise dos resultados

## As cores reduzem fusões

Com redução fixa, repetir uma palavra repete a continuação. Com cores, o caminho também depende da coluna.

As cores não eliminam coincidências. Se duas cadeias chegam à mesma palavra na mesma coluna, compartilham a continuação.

Essas colisões de redução não exigem colisões criptográficas de SHA-256.

## Menos pares, mais reconstrução

De 512×16 para 64×128, em cores, o armazenamento caiu **87,4%** e a consulta ficou **59,5 vezes** mais lenta.

Só as continuações de uma busca que esgota as posições podem exigir $t(t-1)/2$ aplicações de H, além das reduções e reconstruções candidatas.

# Conclusão e limites

Nos cenários medidos, as cores preservaram melhor a recuperação. Aumentar a largura reduziu o arquivo e elevou o tempo de busca.

## Escolha do formato

Tabelas altas favorecem consultas rápidas. Tabelas baixas e largas economizam armazenamento.

Para 7.776 candidatos, busca direta é viável. A utilidade da pré-computação depende da reutilização.

## Alcance do experimento

Uma entrada Diceware por senha, quatro formatos, três sementes e 200 alvos.

Os tempos vêm de uma medição por cenário. Não estimamos variação temporal. O prefixo comum permite reutilizar a tabela, enquanto sais distintos exigem tabelas compatíveis.

# Código e reprodução

## Execução

Python 3.10 ou superior, somente biblioteca padrão.

```
python rainbow_tables.py
python testar_busca.py
```

No repositório, a lista fica em dados/. Ao extrair os anexos do PDF, deixe a lista ao lado dos códigos.

## Apresentação e códigos no GitHub

github.com/oMaDaSa/rainbow_tables/tree/main/apresentacao

A pasta reúne este PDF, rainbow_tables.py e testar_busca.py. As páginas seguintes também contêm os dois códigos completos.

Programa, testes, lista e CSV permanecem anexados ao próprio PDF para reprodução.

---

Fontes: Ivan Sendin, Rainbow Tables, GBC083/UFU, slides 32-36 e 46-47. Arnold G. Reinhold, Diceware Word List, cópia obtida em 16/10/2026.

# rainbow_tables.py (1/8)

O arquivo está inteiro e na ordem de execução. Os números à esquerda não fazem parte do Python.

```
001 """Rainbow Tables - GBC083.
002 Matheus Dantas Santana e Vanessa Dias Ferreira.
003 Execução: python rainbow_tables.py (Python 3.10+, sem pacotes externos).
004 """
005
006 import csv
007 import hashlib
008 import json
009 import random
010 import statistics
011 from pathlib import Path
012 from time import perf_counter
013
014 RAIZ = Path(__file__).resolve().parent
015 LISTA = RAIZ / "diceware.wordlist.asc"
016 if not LISTA.is_file():
017     LISTA = RAIZ.parent / "dados" / "diceware.wordlist.asc"
018 FORMATOS = [(512, 16), (256, 32), (128, 64), (64, 128)]
019 SEMENTES = [17, 43, 91]
020
021 def carregar_palavras(caminho: Path = LISTA) -> list[str]:
022     """Devolver as palavras em ordem crescente do código Diceware."""
023
```

# rainbow_tables.py (2/8)

O arquivo está inteiro e na ordem de execução. Os números à esquerda não fazem parte do Python.

```
024     codigo_para_palavra = {}
025     for linha in caminho.read_text(encoding="utf-8").splitlines():
026         campos = linha.split()
027         if len(campos) != 2:
028             continue
029         codigo, palavra = campos
030         if len(codigo) != 5 or any(digito not in "123456" for digito in codigo):
031             continue
032         if codigo in codigo_para_palavra:
033             raise ValueError(f"Código Diceware duplicado: {codigo}")
034         codigo_para_palavra[codigo] = palavra
035 
036     palavras = [codigo_para_palavra[codigo] for codigo in sorted(codigo_para_palavra)]
037 
038     if len(set(palavras)) != 7776:
039         raise ValueError("A lista deve conter 7.776 códigos e palavras distintos")
040     if palavras[0] != "a" or palavras[-1] != "@":
041         raise ValueError("A lista deve começar em 'a' e terminar em '@'")
042 
043     return palavras
044 
045 
046 def calcular_hash(senha: str) -> str:
```

# rainbow_tables.py (3/8)

O arquivo está inteiro e na ordem de execução. Os números à esquerda não fazem parte do Python.

```
047     return hashlib.sha256(b"GBC083" + senha.encode("utf-8")).hexdigest()
048
049
050 def reduzir(h: str, coluna: int, palavras: list[str], modo: str = "cores") -> str:
051     """Transformar um hash em uma palavra da lista."""
052
053     if coluna < 0 or not palavras or modo not in {"cores", "fixa"}:
054         raise ValueError("Exigidos coluna >= 0, lista não vazia e modo cores ou fixa")
055
056     cor = coluna if modo == "cores" else 0
057
058     mistura = hashlib.sha256(f"{cor}:{h}".encode("utf-8")).hexdigest()
059     posicao = int(mistura, 16) % len(palavras)
060
061     return palavras[posicao]
062
063 # Uma linha guarda duas PALAVRAS, não o hash final.
064 Pares = list[tuple[str, str]]
065
066
067 def fim_da_cadeia(inicio: str, palavras: list[str], largura: int, modo: str = "cores") -> str:
068     """Aplicar exatamente largura passos H -> R e devolver p_t."""
069
```

# rainbow_tables.py (4/8)

O arquivo está inteiro e na ordem de execução. Os números à esquerda não fazem parte do Python.

```
070     if largura < 1:
071         raise ValueError("A largura deve ser pelo menos 1")
072
073     atual = inicio
074
075     for coluna in range(largura):
076         h = calcular_hash(atual)
077         atual = reduzir(h, coluna, palavras, modo)
078
079     return atual
080
081
082 def construir(palavras: list[str], altura: int, largura: int, semente: int, modo: str = "cores") -> Pares:
083     """Sortear inícios distintos e devolver os pares início/fim."""
084     if not 1 <= altura <= len(palavras) or largura < 1:
085         raise ValueError("Exigidos 1 <= altura <= tamanho da lista e largura >= 1")
086
087     inicios = random.Random(semente).sample(palavras, altura)
088     pares = []
089
090     for inicio in inicios:
091         fim = fim_da_cadeia(inicio, palavras, largura, modo)
092         pares.append((inicio, fim))
```

# rainbow_tables.py (5/8)

O arquivo está inteiro e na ordem de execução. Os números à esquerda não fazem parte do Python.

```
093
094     return pares
095
096 Indice = dict[str, list[str]]  # final -> TODOS os inícios que chegam nele
097
098
099 def indexar(pares: list[tuple[str, str]]) -> Indice:
100     """Agrupar inícios pelo final; preservar repetições de final."""
101     indice: Indice = {}
102     for inicio, fim in pares:
103         if fim not in indice:
104             indice[fim] = []
105         indice[fim].append(inicio)
106     return indice
107
108
109 def buscar(alvo: str, palavras: list[str], indice: Indice, largura: int,
110            modo: str = "cores") -> str | None:
111     """Testar posições e confirmar o hash ao reconstruir os candidatos."""
112     if largura < 1:
113         raise ValueError("A largura deve ser pelo menos 1")
114
115     for k in range(largura - 1, -1, -1):
```

# rainbow_tables.py (6/8)

O arquivo está inteiro e na ordem de execução. Os números à esquerda não fazem parte do Python.

```
116         hash_atual = alvo
117         for j in range(k, largura):
118             palavra = reduzir(hash_atual, j, palavras, modo)
119             if j < largura - 1:
120                 hash_atual = calcular_hash(palavra)
121 
122         final_candidato = palavra
123         for inicio in indice.get(final_candidato, []):
124             atual = inicio
125             for coluna in range(largura):
126                 h = calcular_hash(atual)
127                 if h == alvo:
128                     return atual
129                 atual = reduzir(h, coluna, palavras, modo)
130 
131     return None
132 
133 def experimentar(formatos=FORMATOS, sementes=SEMENTES,
134                  quantidade=200, saida=None):
135     """Comparar os formatos usando os mesmos alvos independentes."""
136     palavras = carregar_palavras()
137     alvos = random.Random(2026).sample(palavras, quantidade)
138     hashes = [calcular_hash(p) for p in alvos]
```

# rainbow_tables.py (7/8)

O arquivo está inteiro e na ordem de execução. Os números à esquerda não fazem parte do Python.

```
139     resultados = []
140     for modo in ["cores", "fixa"]:
141         for altura, largura in formatos:
142             for semente in sementes:
143                 print(f"{modo}: {altura} x {largura}, semente {semente}", flush=True)
144                 inicio = perf_counter()
145                 pares = construir(palavras, altura, largura, semente, modo)
146                 construcao_s = perf_counter() - inicio
147                 inicio = perf_counter()
148                 indice = indexar(pares)
149                 indexacao_s = perf_counter() - inicio
150                 tabela_bytes = len(json.dumps(
151                     pares, ensure_ascii=False, separators=(",", ":")
152                 ).encode("utf-8"))
153                 sucessos, falhas = [], []
154                 for alvo in hashes:
155                     inicio = perf_counter()
156                     resposta = buscar(alvo, palavras, indice, largura, modo)
157                     tempo = perf_counter() - inicio
158                     if resposta is not None:
159                         if calcular_hash(resposta) != alvo:
160                             raise AssertionError("A busca retornou uma senha incorreta")
161                         sucessos.append(tempo)
```

# rainbow_tables.py (8/8)

O arquivo está inteiro e na ordem de execução. Os números à esquerda não fazem parte do Python.

```
162                     else:
163                         falhas.append(tempo)
164                 resultados.append({
165                     "modo": modo, "altura": altura, "largura": largura,
166                     "semente": semente, "alvos": quantidade,
167                     "acertos": len(sucessos), "taxa_acerto": len(sucessos) / quantidade,
168                     "construcao_s": construcao_s, "indexacao_s": indexacao_s,
169                     "buscas_s": sum(sucessos) + sum(falhas),
170                     "sucesso_medio_ms": statistics.mean(sucessos) * 1000 if sucessos else "",
171                     "falha_media_ms": statistics.mean(falhas) * 1000 if falhas else "",
172                     "tabela_bytes": tabela_bytes, "finais_distintos": len(indice),
173                 })
174     destino = Path(saida) if saida is not None else RAIZ / "resultados.csv"
175     with destino.open("w", newline="", encoding="utf-8") as arquivo:
176         escritor = csv.DictWriter(arquivo, fieldnames=list(resultados[0]))
177         escritor.writeheader()
178         escritor.writerows(resultados)
179     print(f"Resultados em {destino}")
180     return resultados
181
182
183 if __name__ == "__main__":
184     experimentar()
```

# testar_busca.py (1/2)

Verificações reproduzíveis de hash, índice, candidatos presentes e alvo ausente.

```python
"""Verificações funcionais apresentadas no PDF. Execute: python testar_busca.py"""
from rainbow_tables import (
    calcular_hash, carregar_palavras, construir, indexar, buscar, reduzir,
)

palavras = carregar_palavras()
assert calcular_hash("1234") == (
    "5042f15f93d4f75df9cdff09d062d607eaa0140a930b9023c43ca7cc58609b8f"
)
print("Hash do professor: OK")

assert indexar([("A", "Z"), ("B", "Z")]) == {"Z": ["A", "B"]}
print("Finais repetidos preservados: OK")

consultas = 0
for modo in ["cores", "fixa"]:
    pares = construir(palavras, 8, 5, 17, modo)
    indice = indexar(pares)
    for inicio, _ in pares:
        atual = inicio
        for coluna in range(5):
            alvo = calcular_hash(atual)
            resposta = buscar(alvo, palavras, indice, 5, modo)
```

# testar_busca.py (2/2)

Verificações reproduzíveis de hash, índice, candidatos presentes e alvo ausente.

```
            assert resposta is not None and calcular_hash(resposta) == alvo
            consultas += 1
            atual = reduzir(alvo, coluna, palavras, modo)
print(f"Hashes presentes nas cadeias: {consultas}/{consultas}")

pares = construir(palavras, 4, 1, 17)
inicios = {inicio for inicio, _ in pares}
ausente = next(p for p in palavras if p not in inicios)
assert buscar(calcular_hash(ausente), palavras, indexar(pares), 1) is None
print("Alvo fora da cobertura: None (correto)")
```